

Les dictionnaires

Définition

- chaînes, listes et tuples sont tous des séquences, des suites ordonnées d'éléments
 - il est facile d'accéder à un élément quelconque à l'aide d'un index
- dictionnaires ne sont pas des séquences
 - on peut accéder à un élément du dico à l'aide d'un index spécifique : une **clé**, laquelle pourra être alphabétique, numérique, ou même d'un type composite sous certaines conditions.
 - les éléments mémorisés dans un dictionnaire peuvent être de n'importe quel type : valeurs numériques, chaînes, listes, tuples, dictionnaires, et même fonctions, classes ou instances
 - sont modifiables
 - sont des objets pouvant en contenir d'autres
 - Contrairement aux listes, le dico n'a aucune structure ordonnée

Création d'un dictionnaire

Création d'un dictionnaire de langue

- créer un dictionnaire vide, puis le remplir petit à petit.
- on reconnaît un dictionnaire au fait que ses éléments sont enfermés dans une paire d'accolades

```
>>> dico = {}  
>>> dico['computer'] = 'ordinateur'  
>>> dico['mouse'] = 'souris'  
>>> dico['keyboard'] = 'clavier'  
  
>>> print(dico)  
{'computer': 'ordinateur', 'keyboard': 'clavier', 'mouse': 'souris'}  
>>> print(dico['mouse'])  
souris
```

- les éléments constituent des *paires clé-valeur*.
- dans cet exemple, les clés et les valeurs sont des chaînes

- Un dictionnaire ne peut pas contenir deux clés identiques (la seconde valeur écrase la première)

```
>>>dico={}
```

```
>>>dico["pseudo"]="mdp"
```

```
>>>dico["mot de passe"]="*"
```

```
>>>dico["pseudo"]="password"
```

```
>>>dico
```

```
{'mot de passe': '*', 'pseudo': 'password'}
```

Opérations sur les dictionnaires

```
>>> invent = {'pommes': 430, 'bananes': 312, 'oranges' : 274, 'poires' : 137}
>>> print(invent)
{'oranges': 274, 'pommes': 430, 'bananes': 312, 'poires': 137}
```

```
>>> del invent['pommes']
>>> print(invent)
{'oranges': 274, 'bananes': 312, 'poires': 137}
```

```
>>> print(len(invent))
3
```

```
>>> invent.pop("bananes") #la méthode pop supprime également la clé mais elle renvoie la
                           valeur supprimée
```

312

Test d'appartenance

```
>>> if "pommes" in invent:
...     print("Nous avons des pommes")
... else:
...     print("Pas de pommes. Sorry")
...
Pas de pommes. Sorry
```

Les dictionnaires sont des objets

- La méthode **keys()**
 - renvoie la séquence des *clés* utilisées dans le dictionnaire.
 - cette séquence peut être utilisée telle quelle dans les expressions, ou convertie en liste ou en tuple si nécessaire, avec les fonctions intégrées correspondantes **list()** et **tuple()**

```
>>> print(dico.keys())
dict_keys(['computer', 'mouse', 'keyboard'])
>>> for k in dico.keys(): #on peut mettre que for k in dico mais ajouter keys est mieux
...     print("clé :", k, " --- valeur :", dico[k])
...
clé : computer --- valeur : ordinateur
clé : mouse --- valeur : souris
clé : keyboard --- valeur : clavier
>>> list(dico.keys())
['computer', 'mouse', 'keyboard']
>>> tuple(dico.keys())
('computer', 'mouse', 'keyboard')
```

- La méthode **values()**
 - renvoie la séquence des *valeurs* mémorisées dans le dictionnaire, on peut l'utiliser pour parcourir ou imprimer

```
>>> print(invent.values())
dict_values([274, 312, 137])
for valeur in invent.values():
    print(...)
```

Les dictionnaires sont des objets

- La méthode **items()** :

- extrait du dictionnaire une séquence équivalente de tuples

```
>>> invent.items()
dict_items([('poires', 137), ('bananes', 312), ('oranges', 274)])
>>> tuple(invent.items())
(('poires', 137), ('bananes', 312), ('oranges', 274))
```

- parcours des clés et valeurs simultanément

```
>>>for cle, valeur in invent.items():
```

```
    print ("la clé {} contient la valeur {}".format(cle,valeur))
```

La méthode **copy()** : permet d'effectuer une vraie copie d'un dictionnaire.

- la simple affectation d'un dictionnaire existant à une nouvelle variable crée seulement une nouvelle référence vers le même objet, et non un nouvel objet.

```
>>> stock = invent
>>> stock
{'oranges': 274, 'bananes': 312, 'poires': 137}
```

- si l'on modifie **invent**, alors **stock** est également modifié, et vice-versa (ces deux noms désignent le même objet dictionnaire dans la mémoire de l'ordinateur)

```
>>> del invent['bananes']
>>> stock
{'oranges': 274, 'poires': 137}
```

Pour obtenir une vraie copie (indépendante) : méthode **copy()** :

```
>>> magasin = stock.copy()
>>> magasin['prunes'] = 561
>>> magasin
{'oranges': 274, 'prunes': 561, 'poires': 137}
>>> stock
{'oranges': 274, 'poires': 137}
>>> invent
{'oranges': 274, 'poires': 137}
```


Parcours d'un dictionnaire

- au cours de l'itération, ce sont les *clés* utilisées dans le dictionnaire qui seront successivement affectées à la variable de travail, et non les *valeurs* ;
- l'ordre dans lequel les éléments seront extraits est imprévisible (puisque un dictionnaire n'est pas une séquence).

```
>>> invent = {"oranges":274, "poires":137, "bananes":312}
>>> for clef in invent:
...     print(clef)
...
poires
bananes
oranges
```

- si l'on souhaite effectuer un traitement sur les valeurs, faire appel à la méthode **items()**

```
for clef, valeur in invent.items():
    print(clef, valeur)
...
poires 137
bananes 312
```

Parcours

```
dico = {"fraise":"fruit", "framboise":"fruit", "poireau":"légume",  
"aubergine":"légume", "artichaut":"légume", "abricot":"fruit", }
```

Parcourir un dictionnaire

N.B.: par défaut, le parcours de dico se fait sur les clés

```
print("Parcours standard\n")
```

```
for cle in dico: #un dico se parcourt par ses clés
```

```
    print(cle,dico[cle])
```

Parcourir sur les valeurs

```
print("\n***\nParcours sur valeurs\n")
```

```
for valeur in dico.values():
```

```
    print(valeur)
```

Parcours

```
dico = {"fraise":"fruit", "framboise":"fruit", "poireau":"légume",  
"aubergine":"légume", "artichaut":"légume", "abricot":"fruit", }
```

Parcourir sur les clés et les valeurs

```
print("\n***\nParcourir sur clés et valeurs\n")
```

```
for couple in dico.items():          # par défaut, tuple (clé,valeur)  
    print(couple)
```

```
for cle,valeur in dico.items():      # mais on peut scinder le tuple directement  
    print(cle,valeur)
```

Parcours

```
dico = {"fraise":"fruit", "framboise":"fruit", "poireau":"légume",  
"aubergine":"légume", "artichaut":"légume", "abricot":"fruit", }
```

Parcourir un dictionnaire trié sur les clés

```
print("\n***\nParcours clés triées\n")  
for cle in sorted(dico):  
    print(cle,dico[cle])
```

Trier un dictionnaire

```
>>> sorted({1: 'D', 2: 'B', 3: 'B', 4: 'E', 5: 'A'})  
[1, 2, 3, 4, 5]
```

Rappel : *sorted* s'applique à n'importe quelle donnée

Parcourir un dictionnaire trié sur les valeurs

```
print("\n***\nParcours valeurs triées\n")  
for cle in sorted(dico, key=lambda x:dico[x]):  
    print(cle, dico[cle])
```

Les dictionnaires ne sont pas des séquences

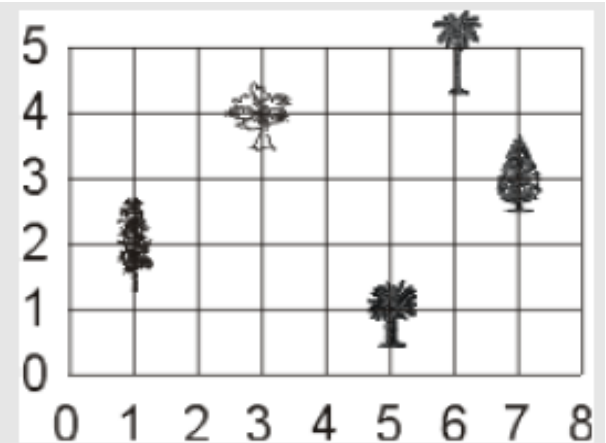
- des opérations comme la concaténation et l'extraction ne peuvent pas s'appliquer
- les dictionnaires se révèlent très précieux pour gérer des ensembles de données où l'on est amené à effectuer fréquemment des ajouts ou des suppressions, dans n'importe quel ordre.
- Ils remplacent avantageusement les listes lorsqu'il s'agit de traiter des ensembles de données numérotées, dont les numéros ne se suivent pas.

```
>>> client = {}  
>>> client[4317] = "Dupond"  
>>> client[256] = "Durand"  
>>> client[782] = "Schmidt"
```

Les clés ne sont pas nécessairement des chaînes de caractères

Répertorier les arbres remarquables situés dans un grand terrain rectangulaire.
=> on utilisera un dictionnaire, dont les clés sont des tuples indiquant les coordonnées **x,y** de chaque arbre :

```
>>> arb = {}  
>>> arb[(1,2)] = 'Peuplier'  
>>> arb[(3,4)] = 'Platane'  
>>> arb[6,5] = 'Palmier'  
>>> arb[5,1] = 'Cycas'  
>>> arb[7,3] = 'Sapin'  
  
>>> print(arb)  
{(3, 4): 'Platane', (6, 5): 'Palmier', (5, 1):  
  'Cycas', (1, 2): 'Peuplier', (7, 3): 'Sapin'}  
  
>>> print(arb[(6,5)])  
palmier
```



- les parenthèses délimitant les tuples sont facultatives

Appeler une clé pas présente dans un dictionnaire

Quand on appelle une clé qui n'est pas présente dans un dictionnaire, on obtient un message d'erreur. Il y a plusieurs manières d'éviter ça :

- tester la présence de la clé dans le dictionnaire avec *if* et *in*

```
if "chocolat" in dico :
```

par défaut, "in" interroge les clés
du dico (pas les valeurs !)

```
    print("C'est dedans !")
```

```
else :
```

```
    print("Pas de ça ici !")
```

Appeler une clé pas présente dans un dictionnaire

Quand on appelle une clé qui n'est pas présente dans un dictionnaire, on obtient un message d'erreur. Il y a plusieurs manières d'éviter ça : **Méthode get**

- Cette méthode retourne une valeur pour une clé donnée. Si la clé est introuvable, on peut donner une valeur à retourner par défaut:

```
Dict.get(key, default=None)
```

Testez :

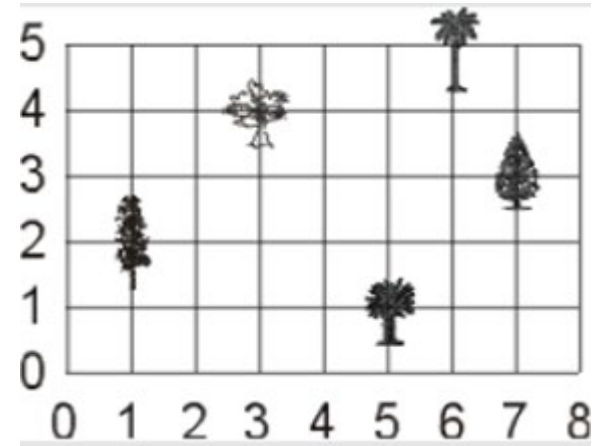
- `dic = {"A":1, "B":2}`
- `print(dic.get("A"))`
- `print(dic.get("C"))`
- `print(dic.get("C","Not Found ! "))`

Appeler une clé pas présente dans un dictionnaire

Quand on appelle une clé qui n'est pas présente dans un dictionnaire, on obtient un message d'erreur. Il y a plusieurs manières d'éviter ça : **Méthode get**

- Utiliser la méthode `.get` (clé à tester, valeur à retourner si la clé n'est pas dans le dico)
- Il donne la réponse s'il y en a, sinon il prend ce que vous l'avez donné (néant dans le cas suivant)

```
>>> arb.get((1,2), 'néant')  
Peuplier  
>>> arb.get((2,1), 'néant')  
néant
```



```
# Utiliser ".get()"  
print(dico.get("chocolat","Rien"))
```

Appeler une clé pas présente dans un dictionnaire

Quand on appelle une clé qui n'est pas présente dans un dictionnaire, on obtient un message d'erreur. Il y a plusieurs manières d'éviter ça :

- gérer l'erreur avec *try* et *except KeyError*

try :

```
print(dico["chocolat"])
```

except KeyError :

```
print("Pas de chocolat dans notre dico...")
```

Récupérer une valeur dans un dictionnaire

```
>>> data = {"name": "Olivier", "age": 30}
```

```
>>> data.get("name")
```

```
'Olivier'
```

```
>>> data.get("adresse", "Adresse inconnue")
```

```
'Adresse inconnue'
```

Créer un dico d'une manière dynamique

```
dico = {  
    "pair":list(),  
    "impair":list(),  
}  
for nombre in range(0,100):  
    if nombre%2 == 0:  
        dico["pair"].append(nombre)  
    else:  
        dico["impair"].append(nombre)  
print(dico["pair"])  
=>  
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22...]
```

Module collections : defaultdict et Counter

- defaultdict : permet de donner une valeur par défaut à une clé. Très pratique quand on remplit automatiquement un dictionnaire qui a des listes pour valeurs !
- Counter : compte automatiquement le nombre d'éléments dans une séquence. Très pratique pour avoir la fréquence des mots d'un texte !

Module collections : Counter

```
from collections import Counter
```

```
# Compter les caractères
```

```
texte = "La la la li li li lu lu lu po po po mu mu mu"
```

```
compteur_caracteres_freq = Counter(texte)
```

```
print(compteur_caracteres_freq)
```

```
print(compteur_caracteres_freq["a"])
```

```
=>
```

```
Counter({' ': 14, 'l': 8, 'u': 6, 'a': 3, 'i': 3, 'p': 3, 'o': 3, 'm': 3, 'L': 1})
```

	14
l	8
u	6
a	3
i	3
p	3
o	3
m	3
L	1

Module collections : Counter

```
from collections import Counter
# Compter les mots
texte = "La la la li li li lu lu lu po po po mu mu mu"
mots = texte.split(" ")
compteur_mot_freq = Counter(mots)
compteur2 = Counter([m.lower() for m in mots])
print(compteur_mot_freq)
print(compteur_mot_freq["la"])
print(compteur2)
=>
Counter({'li': 3, 'lu': 3, 'po': 3, 'mu': 3, 'la': 2, 'La': 1}) 2
Counter({'la': 3, 'li': 3, 'lu': 3, 'po': 3, 'mu': 3})
```

Expression lambda

Trier un dictionnaire

Trier un dictionnaire sur les valeurs est particulièrement utile quand on s'intéresse au vocabulaire d'un texte. Cela permet en effet de trier les mots par ordre de fréquence.

Pour cela, il faut utiliser l'option `key` de `sorted()`. On peut utiliser cette option en association avec une expression `lambda`. Une expression `lambda` (cf. <https://docs.python.org/fr/3/reference/expressions.html#lambda>) correspond à une sorte de fonction très simple, du type `f(x)`. Autrement dit, pour une valeur donnée, on renvoie une autre valeur.

Par exemple, quand on veut trier un dictionnaire sur ses valeurs, pour chaque clé du dictionnaire, on va renvoyer la valeur associée. C'est sur cette valeur (et non sur la clé) qu'on va se baser pour trier le dico.

Module collections : Counter

```
from collections import Counter  
  
# Compter les caractères  
  
texte = "La la la li li li lu lu lu po po po mu mu mu"  
compteur_caracteres_freq = Counter(texte)  
  
for caractere in sorted(compteur_caracteres_freq,  
key=lambda x:compteur_caracteres_freq[x],  
reverse=True):  
    print(caractere,  
compteur_caracteres_freq[caractere])
```

	14
l	8
u	6
a	3
i	3
p	3
o	3
m	3
L	1

Travail à faire

- Faire un dictionnaire dont les clés sont des tuples contenant la lettre et le chiffre identifiant la case, auxquelles on associe comme valeurs le nom des pièces.



Travail à faire

- Afficher tous les éléments d'un dictionnaire selon un ordre croissant des clés
- Construire un dictionnaire inversé pour lequel les valeurs seront les clés et réciproquement.